

Contagem de Primos com OpenMP

Eugênio Vitor Lopes dos Santos

DCA3703 – Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

Implementar um programa em C que conte quantos números primos existem entre 2 e um valor máximo N . Paralelizar o laço principal com `#pragma omp parallel for` sem alterar a lógica original. Comparar o tempo de execução e os resultados das versões sequencial e paralela. Observar diferenças no resultado e no desempenho. Refletir sobre correção e distribuição de carga.

2 Fundamentação teórica

2.1 Paralelismo de laços

O OpenMP distribui iterações de um laço `for` entre threads. A diretiva `#pragma omp parallel for` cria as threads e divide os índices. O escalonamento padrão (`static`) atribui blocos contíguos de tamanho igual a cada thread.

2.2 Race condition

Uma race condition ocorre quando threads escrevem na mesma variável sem sincronização. A CPU executa `total_primos_paralelo++` em três passos: LOAD (lê da memória), ADD (soma 1 no registrador) e STORE (grava na memória). Se duas threads executam o LOAD ao mesmo tempo, ambas leem o mesmo valor. Ambas somam 1 e gravam o mesmo resultado. O programa perde um incremento.

3 Implementação

O código usa $N = 5.000.000$. A função `eh_primo` verifica divisores ímpares até $\sqrt{n}$. O programa executa duas versões: uma sequencial, que percorre o laço em uma thread, e uma paralela, que adiciona `#pragma omp parallel for` ao mesmo laço. Um script varia o número de threads de 1 até o máximo disponível na máquina.

```

1 #define _POSIX_C_SOURCE 199309L
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <time.h>
6 #include <omp.h>
7
8 #define TOTAL_ELEMENTOS_PADRAO 5000000L
9
10 static double obter_tempo_segundos(void) {
11     struct timespec tempo_atual;
12     clock_gettime(CLOCK_MONOTONIC, &tempo_atual);
13     return (double)tempo_atual.tv_sec + (double)tempo_atual.tv_nsec / 1e9;
14 }
15
16 int eh_primo(long numero) {
17     if (numero <= 1) return 0;
18     if (numero == 2) return 1;
19     if (numero % 2 == 0) return 0;
20
21     for (long divisor = 3; divisor * divisor <= numero; divisor += 2) {
22         if (numero % divisor == 0) return 0;
23     }
24
25     return 1;
26 }
27
28 int main(int argc, char **argv) {
29     long total_elementos = argc > 1 ? atol(argv[1]) :
        TOTAL_ELEMENTOS_PADRAO;
30
31     if (total_elementos <= 0) {
32         fprintf(stderr, "Uso: %s [total_elementos]\n", argv[0]);
33         return 1;
34     }
35
36     int total_threads = omp_get_max_threads();
37
38     // SEQUENCIAL
39     double tempo_sequencial = obter_tempo_segundos();
40     long total_primos_sequencial = 0;
41
42     for (long i = 2; i <= total_elementos; ++i) {
43         if (eh_primo(i)) {
44             total_primos_sequencial++;
45         }
46     }
47     tempo_sequencial = obter_tempo_segundos() - tempo_sequencial;

```

```

48
49 // PARALELA
50 double tempo_paralelo = obter_tempo_segundos();
51 long total_primos_paralelo = 0;
52
53 #pragma omp parallel for
54 for (long i = 2; i <= total_elementos; ++i) {
55     if (eh_primo(i)) {
56         total_primos_paralelo++;
57     }
58 }
59 tempo_paralelo = obter_tempo_segundos() - tempo_paralelo;
60
61 printf("N=%ld threads=%d seq_time=%.9f par_time=%.9f seq_count=%ld
62       par_count=%ld\n",
63       total_elementos, total_threads,
64       tempo_sequencial, tempo_paralelo,
65       total_primos_sequencial, total_primos_paralelo);
66
67 return 0;
68 }

```

Listing 1: tarefa05.c

O arquivo é compilado com GCC, `-fopenmp` e `-lm`. O programa recebe o limite superior N como argumento. A medição usa `CLOCK_MONOTONIC`, que não é afetado por alterações no relógio do sistema.

```

1 gcc -std=c11 -Wall -Wextra -fopenmp tarefa05.c -o tarefa05 -lm
2
3 OMP_NUM_THREADS=1 ./tarefa05 5000000
4 OMP_NUM_THREADS=28 ./tarefa05 5000000

```

Listing 2: Compilação e execução dos testes

4 Resultados e Discussão

A versão sequencial contou 348.513 primos em todas as execuções. A versão paralela conta errado: com 2 threads perdeu 245 incrementos, com 28 threads perdeu mais de 25.000. Cada thread escreve no mesmo endereço de memória ao mesmo tempo. O processador aceita os writes e o programa termina sem erro, mas o resultado final está errado.

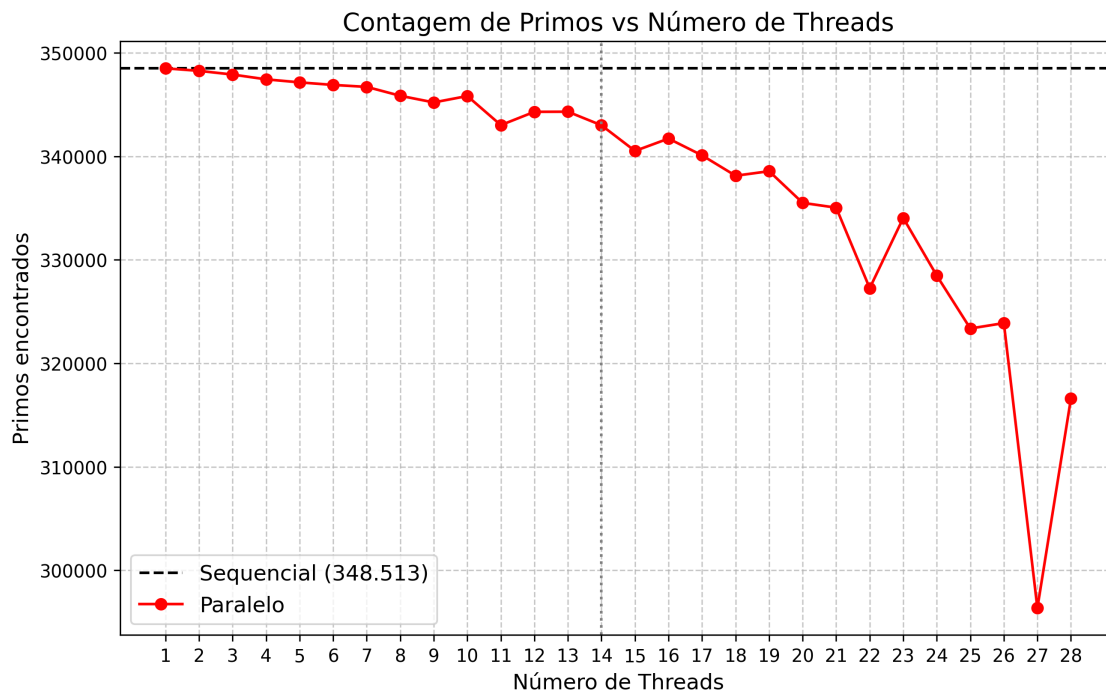


Figure 1: Contagem de primos retornada pela versão paralela. A linha tracejada é o valor correto (348.513). Quanto mais threads, mais incrementos se perdem.

A Figura 1 mostra: com 1 thread a contagem bate. A partir de 2 threads começa a cair. Com 28 threads o programa reporta 323.893 primos, 7% a menos. O programa compila e roda sem warnings e imprime um número errado. O compilador e o runtime não avisam. Só descobre o bug quem compara com a versão sequencial.

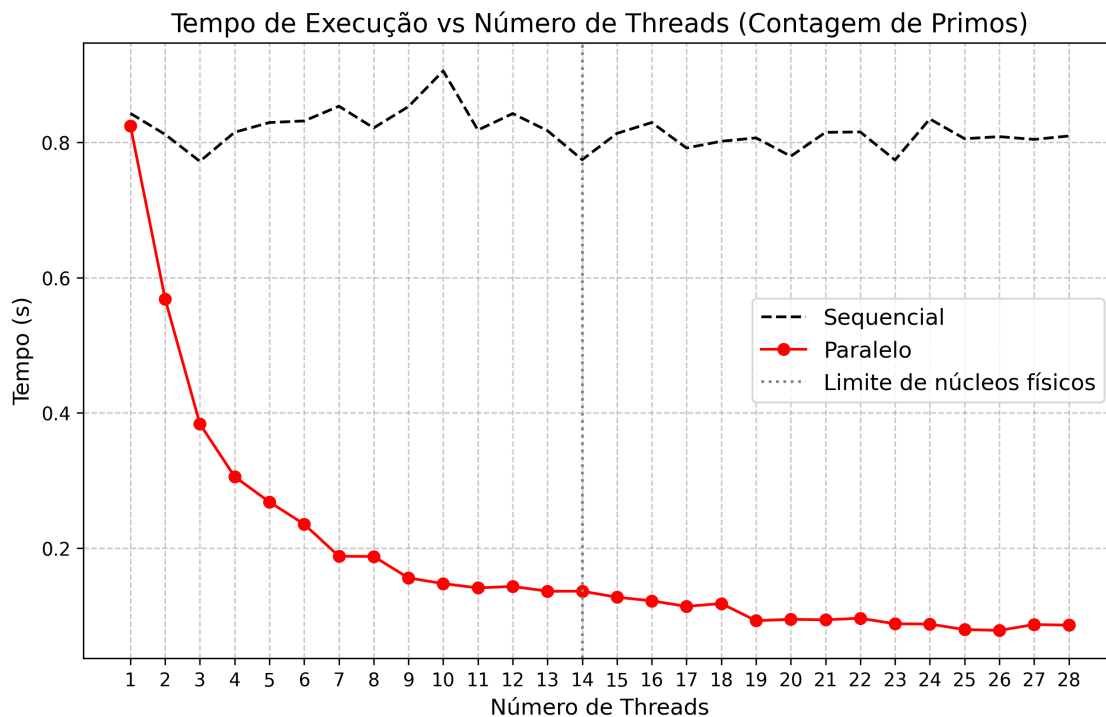


Figure 2: Tempo de execução sequencial vs paralelo. A versão paralela fica mais rápida, mas retorna contagens incorretas.

A Figura 2 mostra o tempo caindo. A versão sequencial mantém $\approx 0,8s$. A paralela cai de 0,57s (2 threads) para 0,086s (28 threads). O escalonamento estático do OpenMP divide os índices em blocos contíguos, mas primos maiores demandam mais divisões. A thread que recebe o bloco final gasta mais tempo, e as primeiras terminam e ficam ociosas esperando a última. O ganho de velocidade é real: 28 threads rodam 6,5x mais rápido que 1 thread. O problema não é velocidade, é correção.

5 Conclusão

O `#pragma omp parallel for` distribui iterações entre threads sem mudar a lógica do laço. Múltiplas threads escrevem na mesma variável sem sincronização, o que causa perda de incrementos. A race condition não gera erro visível: o programa compila, roda e imprime um número errado. O tempo de execução cai com mais threads, mas sem sincronização a contagem cai junto.